Exam Problem: Predicting Ad Clicks for Targeted Email Campaigns

Prompt

We operate an email marketing startup that assists clients, such as beauty e-commerce companies like Sephora, in sending customized email advertisements to targeted customers who are likely to have an interest in specific products. Our goal is to assess the effectiveness of our email campaigns by measuring their impact on website traffic and, ultimately, sales growth.

Your task in this project is to develop a predictive model that determines the likelihood of a customer clicking on an advertisement, based on provided datasets.

Datasets

Available Datasets:

1. user_features.csv: Contains customer data, including demographic information, preferred product categories, and purchase history.
2. product_features.csv: Provides product information, including category, sale status, customer ratings, and other relevant details.
3. click_history.csv: Tracks historical ad clicks, indicating whether a user clicked on an advertisement for a particular product. For example, in the first row, user 104863 did not click on the ad for product 1350.

Using these datasets, design and implement a model that predicts the probability of a click, thereby providing insights into campaign effectiveness and helping optimize future email targeting strategies.

In [1]:
```
import pandas as pd
import os
path = 'C:\\Users\\HP\\Desktop\\DAAN862'
os.chdir(path)

user_features = pd.read_csv('user_features.csv')
product_features = pd.read_csv('product_features.csv')
click_history = pd.read_csv('click_history.csv')
```

Question 1

Explore the basic information of the datasets. Use graphics if necessary. (20 points)

Dataset 1: User Features

To get a good idea of what we are working with, a good first step is to call the head function.

In [2]:
```
import numpy as np
user_features.head()
```

Out[2]:

	user_id	number_of_clicks_before	ordered_before	personal_interests
0	104939	2	True	['body', 'makeup', 'nail', 'hand', 'foot', 'me...
1	101562	2	True	['men_skincare', 'men_fragrance', 'tools', 'sk...
2	102343	2	True	['tools', 'makeup', 'foot', 'nail']
3	106728	5	True	['hand', 'men_skincare']
4	107179	0	True	['makeup', 'body', 'skincare', 'foot', 'men_sk...

Next we will check the datatypes, and get some more information.

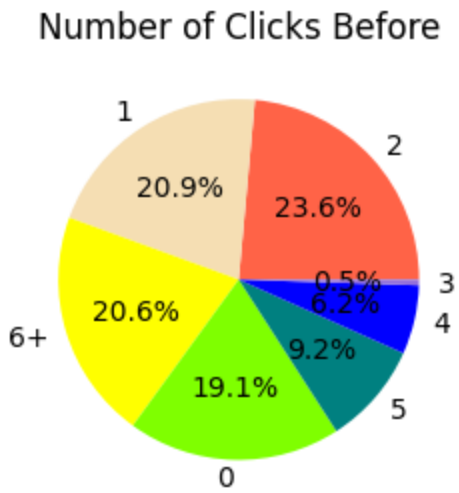
In [3]:
```
user_features.dtypes
```

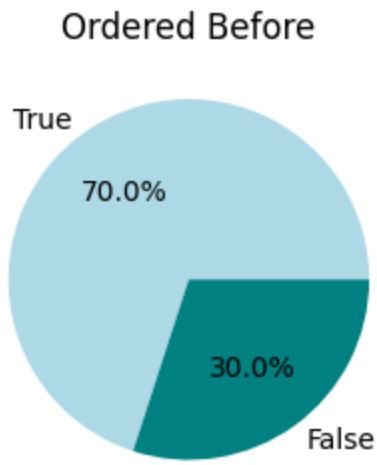
Out[3]:
```
user_id                     int64
number_of_clicks_before    object
ordered_before               bool
personal_interests         object
dtype: object
```

Some Statistics:

In [4]:
```
import matplotlib.pyplot as plt

plt.subplot(1,2,1)
counts = user_features['number_of_clicks_before'].value_counts()
plt.pie(counts.values, labels=counts.index, autopct='%1.1f%%',
       colors=['tomato','wheat','yellow','chartreuse','teal',
               'blue','mediumpurple','violet'])
plt.title('Number of Clicks Before')
plt.subplot(1,2,2)
ordered_counts = user_features['ordered_before'].value_counts()
plt.pie(ordered_counts.values, labels=ordered_counts.index, autopct='%1.1f%%',
       colors=['lightblue','teal'])
plt.title('Ordered Before')
plt.show()
```





In [5]:
```
user_features[['user_id']].describe() #the numerical column
```

Out[5]:

	user_id
count	12000.00000
mean	105999.50000
std	3464.24595
min	100000.00000
25%	102999.75000
50%	105999.50000
75%	108999.25000
max	111999.00000

Dataset 2: Product Features

Similar to the last dataset, we check the first few rows first

In [6]:
```
product_features.head()
```

Out[6]:

	product_id	category	on_sale	number_of_reviews	avg_review_score
0	1134	tools	False	101	3.349452
1	1846	skincare	False	111	5.000000
2	1762	fragrance	False	220	4.882706
3	1254	hair	True	446	5.000000
4	1493	body	True	513	-1.000000

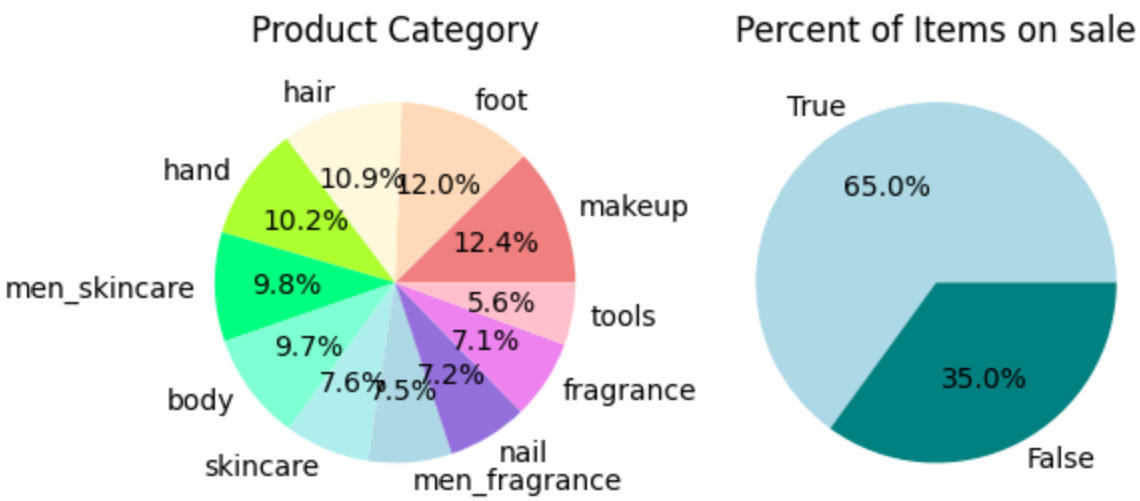
Then data types:

In [7]:
```
product_features.dtypes
```

Out[7]:
```
product_id             int64
category              object
on_sale                 bool
number_of_reviews      int64
avg_review_score     float64
dtype: object
```

Some statistics:

In [8]:
```
plt.subplot(1,2,1)
counts = product_features['category'].value_counts()
plt.pie(counts.values, labels=counts.index, autopct='%1.1f%%',
       colors=['lightcoral','peachpuff','cornsilk','greenyellow',
               'springgreen','aquamarine','paleturquoise','lightblue',
              'mediumpurple','violet','pink'])
plt.title('Product Category')
plt.subplot(1,2,2)
ordered_counts = product_features['on_sale'].value_counts()
plt.pie(ordered_counts.values, labels=ordered_counts.index, autopct='%1.1f%%',
       colors=['lightblue','teal'])
plt.title('Percent of Items on sale')
plt.show()
```



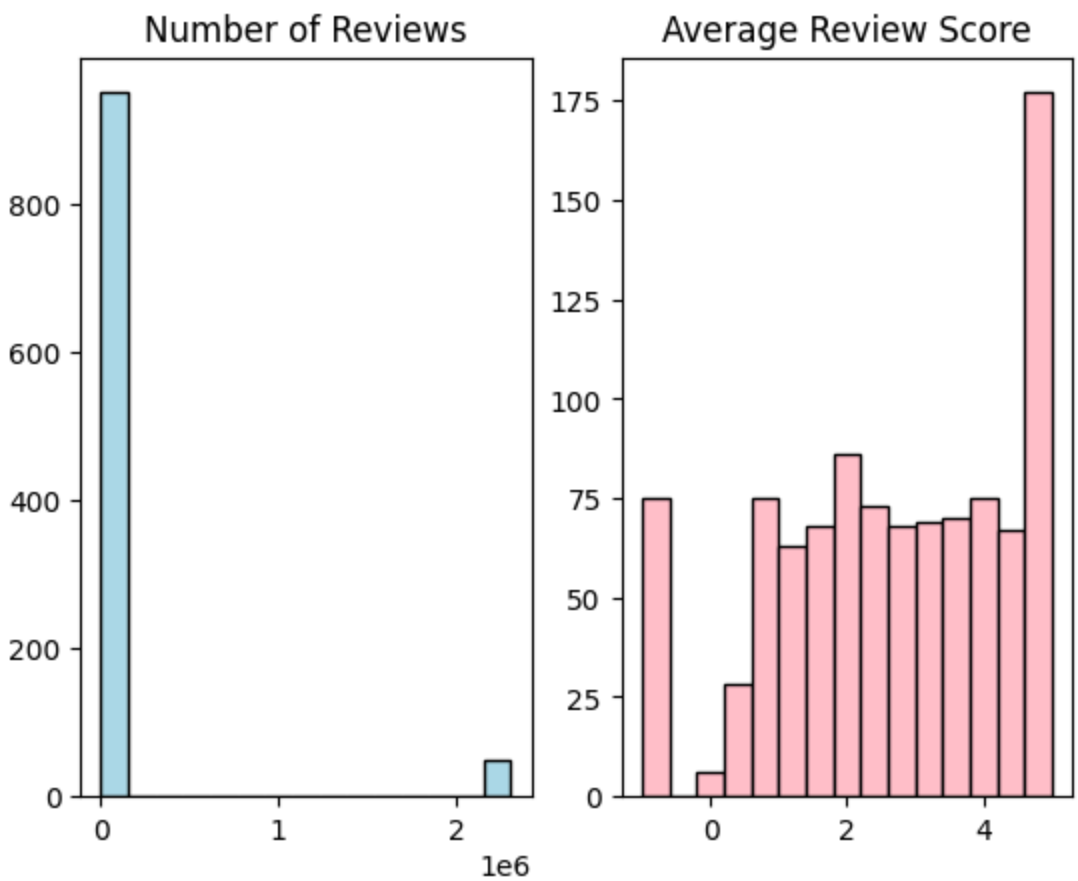
In [9]:
```
# Get all numerical columns
product_features[['product_id','number_of_reviews','avg_review_score']].describe()
```

Out[9]:

	product_id	number_of_reviews	avg_review_score
count	1000.000000	1.000000e+03	1000.000000
mean	1499.500000	1.157725e+05	2.660656
std	288.819436	5.028997e+05	1.741875
min	1000.000000	6.600000e+01	-1.000000
25%	1249.750000	2.570000e+02	1.428969
50%	1499.500000	4.710000e+02	2.769397
75%	1749.250000	7.042500e+02	4.180860
max	1999.000000	2.307390e+06	5.000000

In [10]:

```
plt.subplot(1,2,1)
plt.hist(product_features.number_of_reviews,15,color='lightblue',
        edgecolor='k')
plt.title('Number of Reviews')
plt.subplot(1,2,2)
plt.hist(product_features.avg_review_score,15,color='pink',
        edgecolor='k')
plt.title('Average Review Score')
plt.show()
```



Clearly the number of reviews is generally very low, but there are several items that have many more reviews than everything else. Additionally review scores are fairly evenly distributed with 5 stars being the most common average. Both of these features have quirks that will be dealt with later, including the uneven distribution of reviews, and the existance of review scores being -1.

Dataset 3: Click History

Again, start with the first few rows:

In [11]:
```
click_history.head()
```

Out[11]:

	user_id	product_id	clicked
0	104863	1350	False
1	108656	1321	True
2	100120	1110	False
3	104838	1443	True
4	107304	1397	True

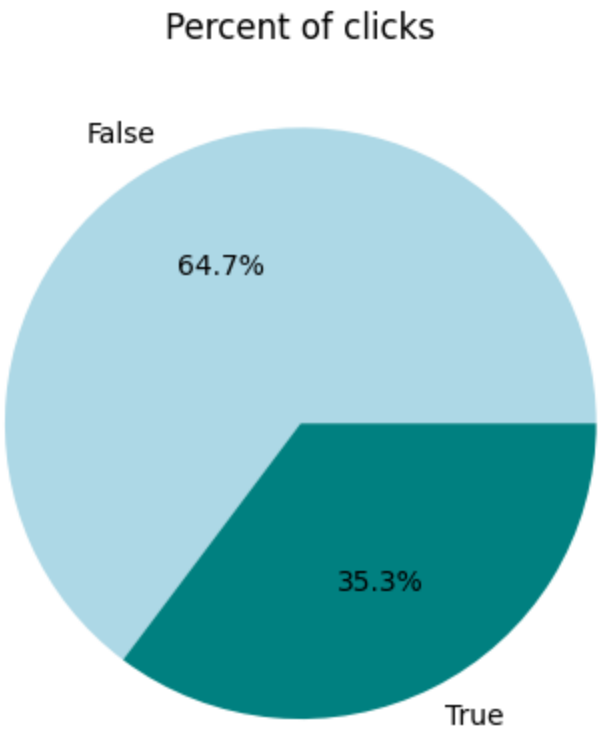
This dataset is fairly straightforward, and simple to display statistics of:

In [12]:
```
click_history[['user_id','product_id']].describe()
```

Out[12]:

	user_id	product_id
count	35990.000000	35990.000000
mean	106017.080161	1500.232898
std	3483.480090	288.101984
min	100001.000000	1000.000000
25%	102976.500000	1250.000000
50%	106060.000000	1503.000000
75%	109049.000000	1749.000000
max	111999.000000	1999.000000

In [13]:
```
clicked_counts = click_history['clicked'].value_counts()
plt.pie(clicked_counts.values, labels=clicked_counts.index, autopct='%1.1f%%',
        colors=['lightblue','teal'])
plt.title('Percent of clicks')
plt.show()
```



Question 2: Data Cleaning and Preprocessing

Clean and preprocess the datasets (such as missing values, outliers, dummy, merging, etc.). (30 points)

Dataset 1: User Features

First, replace all missing values.

In [14]:
```
for i in user_features.columns:
    print(i,':', user_features[i].isnull().values.sum())
```

```
user_id : 0
number_of_clicks_before : 500
ordered_before : 0
personal_interests : 0
```

Since number_of_clicks_before is mostly numerical, it makes sense to replace all missing values with 0.

In [15]:
```
user_features = user_features.fillna(0)
```

Every row should have a unique user id, next we check that.

In [16]:
```
user_features['user_id'].duplicated().any()
```

Out[16]: False

Additionally, we will check that everything in the personal_interests column is a valid product category.

In [17]:
```
product_categories = list(product_features['category'].value_counts().index)
for i in user_features['personal_interests']:
    assert list(i) <= product_categories, "interest isn't a category" #assert that
```

Since no errors are raised, our data is well formatted

Dataset 2: Product Features

Check for missing values to replace:

In [18]:
```
for i in product_features.columns:
    print(i,':', product_features[i].isnull().values.sum())
```

```
product_id : 0
category : 0
on_sale : 0
number_of_reviews : 0
avg_review_score : 0
```

Next we make sure every product ID is unique:

In [19]:
```
product_features['product_id'].duplicated().any()
```

Out[19]: False

We don't need to check all the categories because that was checked earlier in the pie chart, same with the on_sale.

The number of reviews category does have some outliers but that is to be expected because some items will be much more popular than others.

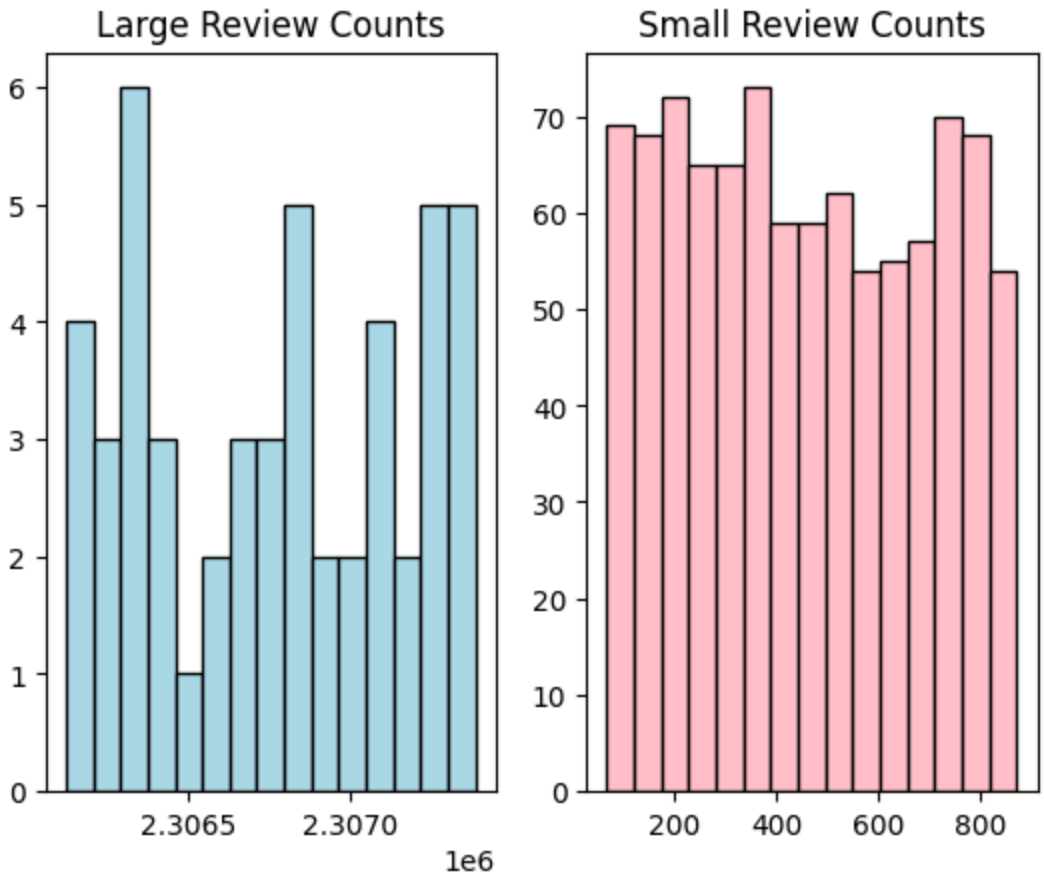
Some items have an average review score of -1. It does not make sense since normally reviews are between 0 and 5, and it does not represent products that have not been reviewed since the minimum reviews on an item is 66. Setting this value to 0 leads to better predictive ability in the model than leaving it as -1.

In [20]:
```
product_features['avg_review_score'] = product_features['avg_review_score'].replace
```

The other issue is the distribution in number of reviews is very large with broadly 2 categories. We will seperate these categories to look at them more closely.

In [21]:
```
large_reviews= product_features[product_features['number_of_reviews'] >= 2000]
small_reviews= product_features[product_features['number_of_reviews'] < 2000]
```

```
plt.subplot(1,2,1)
plt.hist(large_reviews['number_of_reviews'],15,color='lightblue',
        edgecolor='k')
plt.title('Large Review Counts')
plt.subplot(1,2,2)
plt.hist(small_reviews['number_of_reviews'],15, color='pink',
        edgecolor='k')
plt.title('Small Review Counts')
plt.show()
```



Clearly the data is very bimodal. The products that have a large review count have around 2,300,000 reviews, while all the others have 50-900 reivews with nothing in between. This makes the data hard to work with as a useful metric for a learning algorithm. One solution could be to take the base-10 logarithm of the data so the network can more easily distinguish the data. Additionally to not lose the clearly distinguished nature of the review scores, an additional feature is added to indicate if it was initially a large value or a small one.

In [22]:
```
product_features['number_of_reviews'] = np.log10(product_features['number_of_review
product_features.loc[product_features['number_of_reviews'] >= 2000, 'number_of_revi
product_features.loc[product_features['number_of_reviews'] < 2000, 'number_of_revie
product_features.head()
```

Out[22]:

	product_id	category	on_sale	number_of_reviews	avg_review_score	number_of_reviews_
0	1134	tools	False	2.004321	3.349452	
1	1846	skincare	False	2.045323	5.000000	
2	1762	fragrance	False	2.342423	4.882706	
3	1254	hair	True	2.649335	5.000000	
4	1493	body	True	2.710117	0.000000	

Dataset 3: Click History

Replace missing values:

In [23]:
```
for i in click_history.columns:
    print(i,':', click_history[i].isnull().values.sum())
```

```
user_id : 0
product_id : 0
clicked : 0
```

This category does not need more formatting

Data Restructuring

For a learning algorithm to run well, the categorical data must be converted to numerical data. We will start with the personal interests column of user features and convert that into 11 boolean columns. Additionally, we will convert the boolean to 0,1 and the 6+ clicks to 6.

In [24]:
```
import ast #this module lets us interpret the string as a list
user_features_new = user_features
user_features_new['personal_interests'] = user_features['personal_interests'].apply
dummies = pd.get_dummies(user_features_new['personal_interests'].explode()).groupby
user_features_dummy = user_features[['user_id','number_of_clicks_before','ordered_b
user_features_dummy['ordered_before'] = user_features_dummy['ordered_before'].astyp
user_features_dummy['number_of_clicks_before'] = user_features_dummy['number_of_cli
user_features_dummy['number_of_clicks_before'] = user_features_dummy['number_of_cli
user_features_dummy.dtypes #to confirm all categories are numeric
```

Out[24]:
```
user_id                    int64
number_of_clicks_before    int32
ordered_before             int32
body                       int64
foot                       int64
fragrance                  int64
hair                       int64
hand                       int64
makeup                     int64
men_fragrance              int64
men_skincare               int64
nail                       int64
skincare                   int64
tools                      int64
dtype: object
```

In [25]:
```
user_features_dummy.head()
```

Out[25]:

	user_id	number_of_clicks_before	ordered_before	body	foot	fragrance	hair	hand	m
0	104939	2	1	1	1	1	1	1	
1	101562	2	1	1	0	0	0	0	
2	102343	2	1	0	1	0	0	0	
3	106728	5	1	0	0	0	0	1	
4	107179	0	1	1	1	1	1	0	

Next we need to similarly restructure the product information into purely numerical values

In [26]:
```
product_features_new = product_features
# this converts the category into 0-10 inclusive
product_features_new['category'] = pd.factorize(product_features_new['category'])[0
product_features_new['on_sale'] = product_features_new['on_sale'].astype(int)
product_features_new.dtypes
```

Out[26]:
```
product_id                  int64
category                    int64
on_sale                     int32
number_of_reviews         float64
avg_review_score          float64
number_of_reviews_mode    float64
dtype: object
```

In [27]:
```
product_features_new.head()
```

Out[27]:

	product_id	category	on_sale	number_of_reviews	avg_review_score	number_of_reviews_
0	1134	0	0	2.004321	3.349452	
1	1846	1	0	2.045323	5.000000	
2	1762	2	0	2.342423	4.882706	
3	1254	3	1	2.649335	5.000000	
4	1493	4	1	2.710117	0.000000	

The final step is replacing the product id and user id in the final dataset with the corresponding row in the other data sets.

In [28]:

```
click_history['clicked'] = click_history['clicked'].astype(int)
df_combined_user = pd.merge(click_history,user_features_dummy,on='user_id')
df_combined_product = pd.merge(df_combined_user,product_features_new,on='product_id
df_combined = df_combined_product.drop(columns=['user_id','product_id'])
df_combined
```

Out[28]:

	clicked	number_of_clicks_before	ordered_before	body	foot	fragrance	hair	hand
0	0	2	1	1	0	0	1	
1	1	1	1	1	0	1	1	
2	0	0	1	1	1	1	1	
3	1	0	1	1	1	1	1	
4	1	5	1	0	1	1	1	
...	...	...	...	...	...	...	...	
35985	0	1	0	1	1	1	1	
35986	1	1	0	0	0	0	0	
35987	0	5	0	0	0	1	0	
35988	1	2	0	1	1	1	1	
35989	1	0	0	1	1	1	1	

35990 rows × 19 columns

This dataset can now be used to train a predictive model.

Question 3: Model Development, Evaluation and Comparison

Your task is to build, train, optimize, and evaluate a predictive model of your choice. You have the freedom to select any model, including those not covered in this course. However,

you must clearly justify your choice and demonstrate why it is well-suited to this problem. (50 points)

Two types of models will be used and compared. The first is a neural network. This will be used since the dataset is quite large and there may be complex relationships between the many variables that may be more effectively captured by a neural network.

Secondly, a decision tree will be used because most of the data is categorical in nature and the end decision is a binary choice which a setting in which a decision tree ought to perform well.

In [29]:
```
from sklearn.preprocessing import MinMaxScaler
from sklearn import neural_network
from sklearn.model_selection import train_test_split
import sklearn.metrics as metrics
from sklearn.model_selection import cross_val_score

x,y = df_combined.loc[:, df_combined.columns != 'clicked'], df_combined.clicked
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.3, random_state=0)

train,test = train_test_split(df_combined, test_size=.3, random_state=0)
scaler = MinMaxScaler()
scaler.fit(train)
MinMaxScaler(copy=True, feature_range=(0,1))
train = scaler.transform(train)
test = scaler.transform(test)
x_train_scale = train[:,1:]
x_test_scale = test[:,1:]
y_train_scale = train[:,0]
y_test_scale = test[:,0]
nn_model = neural_network.MLPRegressor(10, activation='logistic',max_iter=2000, ran
nn_model.fit(x_train_scale,y_train_scale)
y_pred = nn_model.predict(x_test_scale)
metrics.mean_absolute_error(y_test_scale,y_pred)
```

Out[29]:
```
0.43523854005046014
```

That base score from the model is decent, but we will attempt to optimize it further.

In [30]:
```
from sklearn.model_selection import GridSearchCV
tuned_parameters = {'hidden_layer_sizes':[20,25,30,35,40],
                    'activation':['relu','logistic']}
nn = neural_network.MLPRegressor(max_iter=2000,random_state=0)
nn_optimizer = GridSearchCV(nn, tuned_parameters,
                            scoring='neg_mean_squared_error',
                            return_train_score=False,
                            verbose=0)
nn_optimizer.fit(x_train_scale,y_train_scale)
nn_best_model = nn_optimizer.best_estimator_
nn_best_model.fit(x_train_scale,y_train_scale)
nn_train_pred_best = nn_best_model.predict(x_test_scale)
```

```
nn_mae = metrics.mean_absolute_error(y_test_scale,nn_train_pred_best)
nn_mse = metrics.mean_squared_error(y_test_scale,nn_train_pred_best)
```

Next we will try a decision tree to compare results

In [31]:
```
from sklearn import tree

tree_model = tree.DecisionTreeRegressor(min_samples_leaf=5, random_state=0)
tree_model.fit(x_train,y_train)
y_pred_tree = tree_model.predict(x_test)
metrics.mean_absolute_error(y_test,y_pred_tree)
```

Out[31]:
```
0.32209228593640926
```

In [32]:
```
tuned_parameters_tree = {'min_samples_leaf':[4,5,6,7],
                    'criterion':['squared_error','friedman_mse']}
dt = tree.DecisionTreeRegressor(random_state=0)
dt_optimizer = GridSearchCV(dt, tuned_parameters_tree,
                            scoring='neg_mean_squared_error',
                            return_train_score=False,
                            verbose=0)
dt_optimizer.fit(x_train,y_train)
dt_best_model = dt_optimizer.best_estimator_
dt_best_model.fit(x_train,y_train)
dt_train_pred_best = dt_best_model.predict(x_test)
dt_mae = metrics.mean_absolute_error(y_test,dt_train_pred_best)
dt_mse = metrics.mean_squared_error(y_test,dt_train_pred_best)
```

In [33]:
```
nn_r2 = metrics.r2_score(y_test_scale,nn_train_pred_best)
dt_r2 = metrics.r2_score(y_test,dt_train_pred_best)
results_df = pd.DataFrame({"Neural Network":[nn_mae,nn_mse,nn_r2],
                           "Decision Tree":[dt_mae,dt_mse,dt_r2],
                          "Difference":[nn_mae-dt_mae,nn_mse-dt_mse,nn_r2-dt_r2]},
                          index=["Mean Abs. Error","Mean Sq. Error","R2 Score"])
results_df
```

Out[33]:

	Neural Network	Decision Tree	Difference
Mean Abs. Error	0.418205	0.322951	0.095253
Mean Sq. Error	0.215762	0.212557	0.003205
R2 Score	0.051834	0.065917	-0.014083

Of the two models, the decision tree has better mean absolute error (MAE), but a roughly equivalent mean squared error (MSE). This indicates that the decision tree is better most of the time, but occasionally is more incorrect which has a greater impact on the MSE than the MAE. Overall the decision tree is a good model to predict the clicks given a set of product information and user information.

An example of 1000 random samples is given below:

In [34]:
```
match_dt, match_nn = 0,0
for i in range(1000):
    sample_row = df_combined.sample(n=1).index
    row_vals = df_combined.iloc[sample_row,1:]
    prediction_dt = round(dt_best_model.predict(row_vals)[0])
    prediction_nn = round(nn_best_model.predict(row_vals.values)[0])
    if prediction_nn < 0:
        prediction_nn = 0 #set to zero if below zero
    elif prediction_nn > 1:
        prediction_nn = 1 #set to one if above one
    actual = df_combined.iloc[sample_row,0].values[0]
    if prediction_dt == actual:
        match_dt += 1
    if prediction_nn == actual:
        match_nn += 1
print(match_dt/10,"% correct from Decision Tree","\n",match_nn/10,"% correct from N
```

```
79.1 % correct from Decision Tree
 63.0 % correct from Neural Network
```

This confirms what was seen in the scoring that the decision tree consistantly out-performs the neural network. This is likely because the neural network is creating continuous predictions which do not necesarily allign well with the binary choice of the target, which is why in the above code, the neural network predictions have an additional check rather than just rounding. Also, it is logical that the problem itself is better suited for a decision tree because of the many categorical variables rather than continuous numerical variables.

In order to use the model, the client would simply have to do a similar transformation of customer ID into customer data, and product ID into product data as done above in order to get a prediction of their chance of clicking on the ad for the specified product.